FACULTY OF MECHATRONICS,
INFORMATICS AND
INTERDISCIPLINARY STUDIES TUL

Term paper

Solving the Multidimensional Knapsack Problem: Brute-Force Search and Greedy Heuristics

Author: **Bc. Petr Boháč**

Liberec 2025

CONTENTS

LIST OF IMAGES

1 INTRODUCTION

As a subject of this assignment, an arbitrary NP problem was supposed to be chosen and implemented using a naive brute-force approach. The second part of the assignment was to source or implement a more optimized heuristic algorithm for the same problem and compare the results of both approaches.

The assignment recommended two options - the *Knapsack Problem* and the *Necklace Splitting Problem*. I ended up going with the former[1], as it was simple enough for my monkey brain to comprehend. The conventional problem didn't seem interesing enough on its own though, so I decided to add a little twist to it and make the constraints (the weights) multidimensional - specifically, I went for three distinct weight dimensions, transforming the problem into a *Multidimensional Knapsack Problem* (if that's even a thing).

Both the algorithms were implemented in Python and I specifically opted out of using any concurrency or parallelism, even though the brute-force approach could have greatly benefited from it. Although students are generally expected to source the optimized heuristic algorithm online, I, in my infinite wisdom, decided to complicate the problem by adding multiple dimensions. As a result, I ended up having to implement a simple greedy heuristic myself (with the help of what Boris Johnson likes to call "Chat Chippy Tea"). This is more thoroughly explained in Section 2.

For data visualization, I used the `matplotlib` library to generate plots comparing the results of both algorithms.

To summarize the results, the brute-force approach unsurprisingly outperformed the greedy heuristic in terms of solution quality, but at the cost of significantly higher computation time, especially as the number of items increased. The greedy heuristic provided a faster solution, but it was at times suboptimal compared to the brute-force approach, which is obviously able to explore the entire solution space allowing it to find the optimal solution.

[1] The research involved a thorough 5-minute read of it's Wikipedia page [1]

2 IMPLEMENTATION

As mentioned in Section 1, the first part of the assignment involved implementing a naive brute-force approach which is supposed to be awfully inefficient to really drive the point home about NP problems. The second part was to implement a more optimized heuristic algorithm to compare the results of both approaches.

2.1 THE PROBLEM

Let us also familiarize ourselves with the problem we are solving. Consider a set of application deployments, each with a priority value and resource requirements in three dimensions: memory, CPU, and storage. The goal is to select a subset of these deployments to maximize the total priority value without exceeding the available resources in any dimension.

```
from collections import namedtuple

Deployment = namedtuple("Deployment", [
    "name", "memory", "cpu", "storage", "priority"
])
```

Listing 1: Data structure for representing an application deployment

2.2 BRUTE-FORCE APPROACH

Coming up with a brute-force algorithm is fairly straightforward as it simply involves generating all possible combinations of items and checking which one yields the highest value without exceeding the knapsack's capacity in any dimension.

The implementation uses the `itertools.combinations` from Python's standard library to generate all possible combinations of items. For each combination, it calculates the total weight in each dimension and the total value. If the combination fits within the knapsack's capacity and has a higher value than the best found so far, it updates the best value and combination.

The previous steps are repeated for all possible combination lengths, from 1 to the total number of items.

The complete implementation of the brute-force approach is shown in Listing 2.

```python
def knapsack_bf(items, capacity):
    best_value = 0
    best_combination = set()

    for r in range(len(items) + 1):
        for combination in itertools.combinations(items, r):
            total_memory = sum(item.memory for item in combination)
            total_cpu = sum(item.cpu for item in combination)
            total_storage = sum(item.storage for item in combination)

            if all((
                total_memory <= capacity[0],
                total_cpu <= capacity[1],
                total_storage <= capacity[2],
            )):
                total_value = sum(item.priority for item in combination)
                if total_value > best_value:
                    best_value = total_value
                    best_combination = combination

    return best_value, best_combination
```

Listing 2: Brute-force implementation (simplified)

2.3 GREEDY HEURISTIC

Prior to writing this, I had no understanding of the term *greedy* in the context of algorithms, so I looked it up:

> A **greedy algorithm** is any algorithm that follows the problem-solving heuristic of making the locally optimal choice at each stage.
>
> — Wikipedia

The term *heuristic* was also a bigbrain term for me. My understanding is that it refers to an approach that is not guaranteed to be optimal or perfect, but is practical and sufficient for reaching an immediate goal. It is more of a method or strategy rather than a strict algorithm.

With all this new knowledge, I pieced together an algorithm that tries multiple different heuristics and picks the best result among them.

As showin in Listing 3, four distinct scoring functions were used to evaluate the items. They are pretty much just **value-to-cost** ratios, one for each of the constraints (memory, CPU, storage), and one that considers the total cost across all dimensions.

```
def score_memory(item):
    return item.priority / item.memory

def score_cpu(item):
    return item.priority / item.cpu

def score_storage(item):
    return item.priority / item.storage

def score_total(item):
    return item.priority / (item.memory + item.cpu + item.storage)
```

Listing 3: Scoring functions used in the greedy heuristic

The greediness of the algorithm comes from sorting the items based on each scoring function and then iteratively adding them to the knapsack as long as they fit within the remaining capacity.

This process obviously does not guarantee an optimal solution, but by trying multiple scoring strategies, it increases the chances of finding a good approximation. This further adds to the greediness of the algorithm by continually seeking the best immediate gain through different perspectives.

The complete implementation of the greedy heuristic is shown in Listing 4.

```
def knapsack(items, capacity):
    items_list = list(items)

    score_functions = [
        score_memory,
        score_cpu,
        score_storage,
        score_total
    ]

    best_value = 0
    best_combination = set()

    for score_fn in score_functions:
        sorted_items = sorted(items_list, key=score_fn, reverse=True)

        total_memory = 0
        total_cpu = 0
        total_storage = 0
        chosen = []

        for item in sorted_items:
            if all(
                (
                    total_memory + item.memory <= capacity[0],
                    total_cpu + item.cpu <= capacity[1],
                    total_storage + item.storage <= capacity[2],
                )
            ):
                chosen.append(item)
                total_memory += item.memory
                total_cpu += item.cpu
                total_storage += item.storage

            progress.update(1)

        total_value = sum(i.priority for i in chosen)

        if total_value > best_value:
            best_value = total_value
            best_combination = set(chosen)

    return best_value, best_combination
```

Listing 4: Greedy heuristic implementation (simplified)

3 RESULTS

As it usually is with brute-force algorithms, the time complexity grows exponentially with the number of items, or more formally:

$\mathrm{Time(n)} = O(2^n)$, where $n = |S|$ is the cardinality of the input item set S

This is because all possible combinations of items need to be evaluated to ensure the optimal solution is found. With each additional item, the number of possible combinations doubles. For example, running the algorithm over a set of 22 items results in a runtime of ~8 seconds, while increasing the set to 23 items results in a runtime of ~16 seconds, and 24 items leads to ~32 seconds, and so on.

In contrast, the greedy heuristic has a linear time complexity:

$\mathrm{Time(n)} = O(n)$, where $n = |S|$ is the cardinality of the input item set S

More specifically, the time complexity for our particular implementation is $O(4n)$, since it evaluates four different scoring functions, each requiring a single pass through the list of items to sort them and another pass to select items based on the sorted order. However, in Big O notation, constant factors are disregarded [2], so it simplifies to $O(n)$.

The time complexities of the two approaches are illustrated in Figure 3. 1, where the exponential growth of the brute-force method is clearly contrasted with the linear growth of the greedy heuristic.

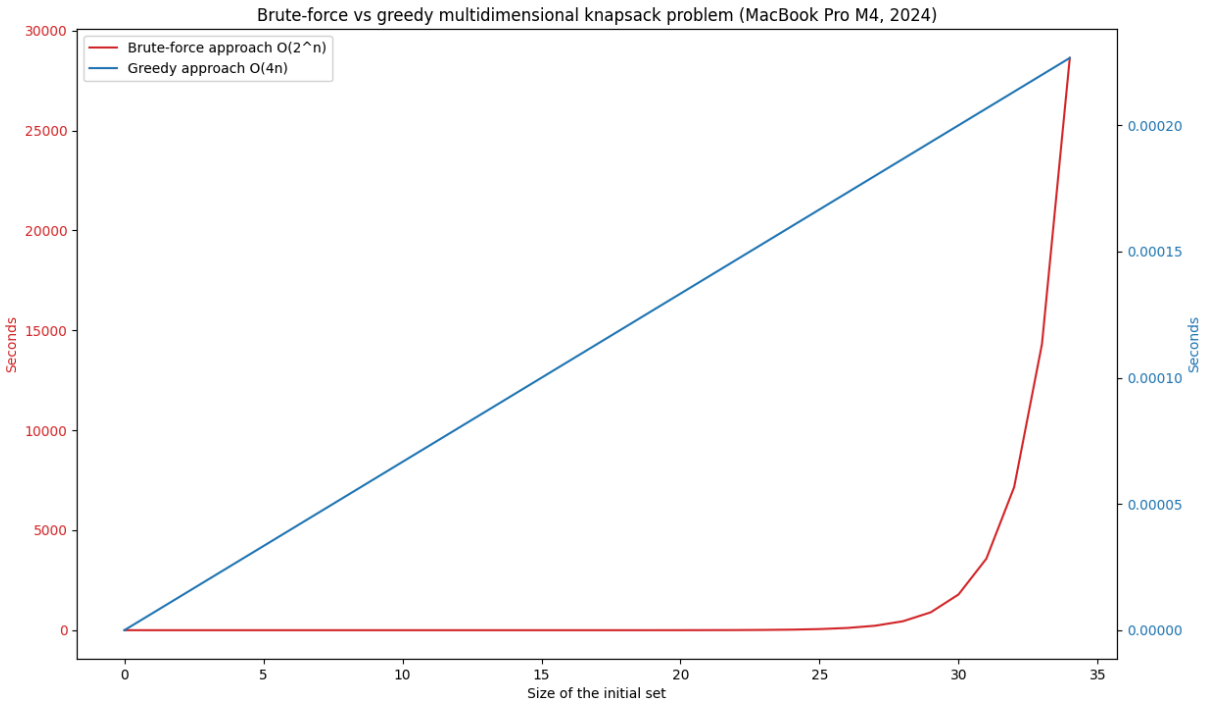

Figure 3. 1: Exponential vs. linear runtime behaviour of BF and greedy knapsack algorithms.

As indicated by the vastly different scales of the two y-axes, the brute-force method becomes computationally infeasible even for moderately large input sizes. In contrast, the greedy heuristic maintains low and predictable running times as the number of items increases, making it suitable for practical use in real-world scenarios.

3.1 Determining the maximum input size for a sub-hour runtime

One of the requirements of the assignment was to determine the maximum input size that allows the algorithms to find the optimal solution within one hour.

This is quite simple for the brute-force approach, as we can employ empirical measurements to extrapolate the maximum input size - a set of 31 items can be processed in under an hour, while a set of 32 items exceeds the time limit (also confirmed by doing the math in Section 3.1.1).

For the optimized algorithm though is so much more performant that the input set (which has to be filled manually) would be unreasonably large to create. Therefore, coming up with a simple mathematical formula to estimate the maximum input size is more appropriate.

A runtime for a given input size n can be expressed as:

$$\mathrm{Time(n)} = \frac{\mathrm{Iter(n)}}{\mathrm{IPS}}$$

Where $\mathrm{Iter(n)}$ is the number of iterations the algorithm performs for input size n, so pretty much just the time complexity - 2^n for brute-force and $4n$ for greedy. The IPS variable is the number of iterations the system can perform per second (or the throughput).

The included progress bar feature of the implementation provides an $\mathrm{ops/sec}$ value allowing us to estimate the IPS values for my system (a 2024 M4 MacBook Pro):

$$\mathrm{IPS(Brute\text{-}force)} \approx 600,000$$

$$\mathrm{IPS(Greedy)} \approx 1,200,000$$

A sub-hour runtime (3600 seconds) means:

$$\mathrm{Iter(n)} \leq 3600 \cdot \mathrm{IPS}$$

3.1.1 BRUTE-FORCE

Considering $\mathrm{Iter(n)} = 2^n$, start with the inequality:

$$2^n \leq 3600 \cdot \mathrm{IPS}$$

Take the base-2 logarithm of both sides:

$$n \leq \log_2(3600 \cdot \mathrm{IPS})$$

Substitute $\mathrm{IPS} = 600000$:

$$n \leq \log_2(3600 \cdot 600000)$$

Compute the value:

$$n \leq \log_2(2160000000)$$

Which evaluates to:

$$n \leq 31.008384...$$

Since n must be an integer:

$$n_{\max}^{(\mathrm{brute})} = \lfloor 31.008384... \rfloor = \mathbf{31}$$

3.1.2 GREEDY

Considering $\mathrm{Iter(n)} = 4n$, start with the inequality:

$$4n \leq 3600 \cdot \mathrm{IPS}$$

Solve for n:

$$n \leq \frac{3600 \cdot \mathrm{IPS}}{4}$$

Substitute $\mathrm{IPS} = 1200000$:

$$n \leq \frac{3600 \cdot 1200000}{4}$$

Simplify:

$$n \leq 900 \cdot 1200000$$

Compute:

$$n \leq 1080000000$$

Thus:

$$n_{\max}^{(\mathrm{greedy})} = \mathbf{1080000000}$$

4 CONCLUSION

Apart from some new terminology, insights into algorithmic efficiency and maybe NP problems as a whole, not much new groundbreaking knowledge was gained from the work put into this assignment. It was obvious from the start that the brute-force approach would yield optimal solutions at the cost of exponential time complexity, while the more optimized approach would trade off some solution quality for significantly improved performance.

As a side note, writing a quick and dirty MS Word document to summarize the results would have been quite enough I'm sure, but I'll have to write my Master's thesis in Typst too, so I figured why not get some practice in now.

BIBLIOGRAPHY

[1] WIKIPEDIA CONTRIBUTORS. Knapsack problem — Wikipedia, The Free Encyclopedia. Online. 2025. Available from: https://en.wikipedia.org/w/index.php?title=Knapsack_problem&oldid=1317349218[Online; accessed 5-December-2025]

[2] TEMPLATETYPEDEF. Answer to "Why is the constant always dropped from big O analysis?". Online. 5 March 2014. [Accessed 6 December 2025]. Available from: https://stackoverflow.com/a/22188943